جلسهٔ ۱۲ — از ویژگی تا یادگیریِ ماشین (Features → Machine Learning)

جزوهٔ درس — هفتهٔ دوازدهم

نگاهِ کلی به این جلسه

این جلسه یک نقطهٔ عطف در درس است. تا هفتهٔ ۱۱، ما هر گام را طراحی می‌کردیم: کدام فیلتر، کدام آستانه، کدام لبه، کدام تبدیل. از این هفته رویکرد عوض می‌شود: به‌جای طراحیِ دستیِ قاعده، به کامپیوتر مثال‌های برچسب‌دار می‌دهیم و می‌گذاریم خودش قاعدهٔ تصمیم را «یاد بگیرد». این همان **یادگیریِ ماشین** (Machine Learning) است. پلی که پردازش تصویرِ کلاسیک را به یادگیریِ ماشین وصل می‌کند، همان مفهومِ آشنای «ویژگی» است: تصویر را به یک بردارِ ویژگی تبدیل می‌کنیم و سپس یک **طبقه‌بند** (classifier) روی این بردارها آموزش می‌دهیم.

یادداشت

تسکِ واقعیِ امروز: طبقه‌بندیِ مدلِ خودروهای ایرانی (پژو ۲۰۶ در برابرِ ۴۰۵ در برابرِ دنا) از روی ویژگی‌های **HOG**، با استفاده از تصاویرِ واقعیِ شما در `E:/data/Vehicle_classes`. این یک مسئلهٔ سختِ واقعی است و — همان‌طور که خواهیم دید — نتیجه‌اش درسِ صادقانه‌ای دربارهٔ محدودیت‌های ویژگیِ دست‌طراحی به ما می‌دهد.

یک تشبیهِ ساده

فرض کنید می‌خواهید به کودکی یاد بدهید سیب را از پرتقال تشخیص دهد. دو راه دارید. راهِ اول (کلاسیک): به او قاعده بدهید — «اگر گِرد و قرمز است سیب، اگر گِرد و نارنجی با پوستِ ناهموار است پرتقال». راهِ دوم (یادگیری): ده‌ها سیب و پرتقالِ برچسب‌دار نشانش دهید و بگذارید خودش قاعده را کشف کند. یادگیریِ ماشین راهِ دوم است. اما یک نکتهٔ ظریف: در راهِ دوم هم باید تصمیم بگیرید کودک به چه چیزی نگاه کند — رنگ؟ شکل؟ بافت؟ این «چه چیزی» همان ویژگی است. امروز ویژگی را ما انتخاب می‌کنیم (HOG) و فقط قاعده را واگذار می‌کنیم؛ در یادگیریِ عمیق، انتخابِ ویژگی هم به ماشین واگذار می‌شود.

۱ خط‌لولهٔ کلاسیکِ شناسایی

تقریباً همهٔ سامانه‌های شناساییِ تصویر تا حدودِ سالِ ۲۰۱۲ این ساختار را داشتند:

۱. **تصویرِ ورودی** →

۲. **استخراجِ ویژگی** (تبدیلِ پیکسل‌ها به یک بردارِ فشرده) →

۳. **طبقه‌بند** (تابعی آموخته‌شده که بردار را به برچسب می‌نگارد) ←

۴. **پیش‌بینی** (۲۰۶ / ۴۰۵ / دنا).

نکتهٔ کلیدی

نکتهٔ تاریخیِ مهم: در این خط‌لوله، ویژگی‌ها دست‌طراحی بودند (توسطِ انسان) و فقط طبقه‌بند یاد گرفته می‌شد. در هفته‌های ۱۳ و ۱۴ خواهیم دید که یادگیریِ عمیق این را تغییر داد: خودِ ویژگی‌ها هم از داده یاد گرفته می‌شوند.

۱.۱ چرا پیکسل‌های خام را مستقیم نمی‌دهیم؟

یک تصویرِ $\times$ برابرِ ۹۲۱۶ عدد است — بسیار بزرگ. مهم‌تر: پیکسل‌های خام **ناوردا** نیستند. اگر همان خودرو را ۵ پیکسل جابه‌جا کنیم یا نور کمی عوض شود، تقریباً همهٔ ۹۲۱۶ عدد تغییر می‌کنند، در حالی که خودرو همان خودروست. یک طبقه‌بند که مستقیم روی پیکسل‌ها کار کند، این جابه‌جاییِ بی‌اهمیت را «تفاوت» می‌بیند.

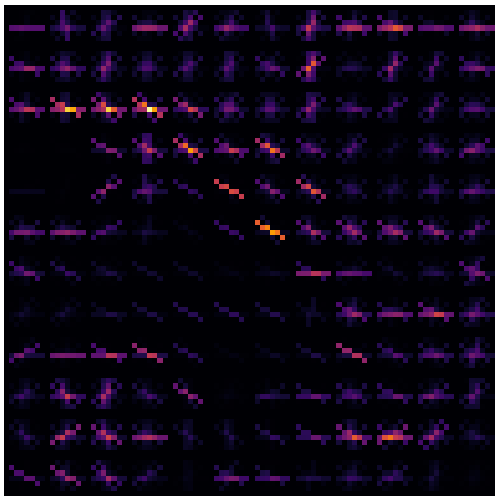
یادداشت

ویژگی‌های خوب (HOG، ORB) هیستوگرامِ رنگ، تصویر را به برداری فشرده، پایدار و معنادار تبدیل می‌کنند. هدفِ نهایی این است که کلاس‌ها در «فضای ویژگی» **جداپذیر** (separable) شوند: چیزهای شبیه به هم نزدیکِ هم بیفتند و چیزهای متفاوت دور از هم.

۲.۱ HOG: هیستوگرامِ گرادیان‌های جهت‌دار

HOG (Histogram of Oriented Gradients) یکی از موفق‌ترین ویژگی‌های دست‌طراحی است. ایده ساده است: تصویر را به سلول‌های کوچک تقسیم کن و در هر سلول، جهتِ گرادیان‌ها را در یک هیستوگرام بشمار. نتیجه یک «امضای شکلیِ» فشرده است.





چپ: یک پژو ۲۰۶ واقعی راست: نقشهٔ HOG آن (جهتِ لبه‌ها در هر سلول)

ریاضیات

چرا HOG برای شکل خوب کار می‌کند؟ خودروها در شکل تفاوت دارند (خطِ سقف، چراغ‌ها، جلوپنجره) و این تفاوت‌ها در جهتِ گرادیان‌ها ثبت می‌شوند. هیستوگرام‌کردن در هر سلول، تحمل نسبت به جابه‌جاییِ کوچک و تغییرِ نور می‌دهد. ترکیبِ HOG + SVM خطی سال‌ها بهترین روش برای تشخیصِ عابر و خودرو بود.

HOG مقدارِ دقیقِ پیکسل‌ها را دور می‌ریزد و فقط نگه می‌دارد که «لبه‌ها به کدام سو اشاره می‌کنند» — و همین جوهرهٔ شکل است.

۳.۱ HOG گام‌به‌گام (شهودِ عددی)

بیایید دقیق‌تر ببینیم HOG چه می‌کند:

۱. گرادیانِ افقی G_x و عمودی G_y هر پیکسل را حساب کن (مثلِ سوبلِ هفتهٔ ۵).

۲. بزرگی $m = \sqrt{G_x^2 + G_y^2}$ و جهت $\theta = \arctan(G_y/G_x)$ را به دست آور.

۳. تصویر را به سلول‌های مثلاً $\times$ تقسیم کن. در هر سلول، جهت‌ها را به ۹ سطلِ (تا درجه) تقسیم و با وزنِ بزرگی جمع کن — این یک هیستوگرامِ ۹تایی می‌دهد.

۴. بلوک‌هایی از $\times$ سلول را کنارِ هم نرمال کن (تا نسبت به تغییرِ نور مقاوم شود).

ریاضیات

مثالِ عددیِ کوچک: اگر در یک سلول بیشترِ لبه‌ها عمودی باشند (مثلِ ستونِ درِ خودرو)، سطلِ نزدیک به $^\circ$ مقدارِ بزرگی می‌گیرد و بقیهٔ سطل‌ها کوچک می‌مانند. پس همان سلول با برداری مثلِ $[\,,\,,\ldots,\,,\ldots]$ توصیف می‌شود که «اینجا لبهٔ عمودیِ قوی هست» را کد می‌کند. کلِ تصویر می‌شود به‌هم‌چسباندنِ این هیستوگرام‌های سلولی.

یادداشت

به همین دلیل HOG «امضای شکل» است: می‌گوید در هر ناحیه لبه‌ها به کدام سو می‌روند، بی‌آنکه به روشناییِ مطلق حساس باشد. برای تصویرِ $\times$ ما، این کار برداری با حدودِ ۹۰۰ عدد تولید کرد.

۲ طبقه‌بند چه چیزی یاد می‌گیرد؟

با داشتنِ بردارهای ویژگیِ برچسب‌دار، طبقه‌بند یک **مرزِ تصمیم** (decision boundary) بینِ کلاس‌ها یاد می‌گیرد. دو طبقه‌بندِ ساده که امروز استفاده می‌کنیم:

- **k نزدیک‌ترین همسایه (k-NN):** برای برچسب‌زدنِ یک نقطه، به k نقطهٔ نزدیکش در دادهٔ آموزش نگاه کن و رأیِ اکثریت را بگیر. عملاً داده را «حفظ» می‌کند.
- **ماشینِ بردارِ پشتیبان (SVM):** مرزی پیدا می‌کند که بیشترین **حاشیه** (margin) را بینِ کلاس‌ها داشته باشد. با کرنلِ RBF می‌تواند مرزهای منحنی هم بسازد.

نکتهٔ کلیدی

هر دو طبقه‌بند به فضای ویژگیِ جداپذیر نیاز دارند. اگر ویژگی‌ها کلاس‌ها را از هم جدا نکنند، هیچ طبقه‌بندی نمی‌تواند معجزه کند — «آشغال ورودی، آشغال خروجی».

۱.۲ k-NN با یک مثالِ عددی

بیایید k-NN را روی یک مثالِ اسباب‌بازی ببینیم. فرض کنید ویژگی‌ها دوبُعدی‌اند و ۴ نمونهٔ آموزش داریم:

ریاضیات

$A(۱,۱) \to$ کلاسِ ۰، $B(۱,۲) \to$ کلاسِ ۰، $C(۵,۵) \to$ کلاسِ ۱، $D(۶,۵) \to$ کلاسِ ۱.

نقطهٔ جدید $P(۲,۲)$ را طبقه‌بندی کنیم با $k = ۳$. فاصله‌ها:

$$d(P,A) = \sqrt{۱+۱} = ۱٫۴۱,\ d(P,B) = ۱,\ d(P,C) = \sqrt{۹+۹} = ۴٫۲۴,\ d(P,D) = ۵$$

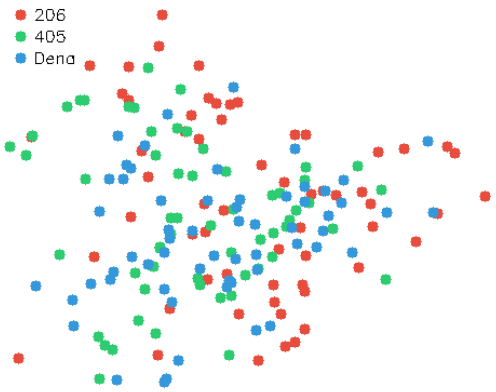
سه همسایهٔ نزدیک: B, A, C با کلاس‌های ۰, ۰, ۱ ← رأیِ اکثریت ← کلاسِ ۰.

نکتهٔ کلیدی

می‌بینید که k-NN هیچ «آموزشی» به‌معنای واقعی ندارد؛ فقط داده را ذخیره می‌کند و در زمانِ پیش‌بینی فاصله‌ها را حساب می‌کند. به همین دلیل آموزشش فوری اما پیش‌بینی‌اش کند است (باید با همهٔ نقاط مقایسه کند). انتخابِ k هم مهم است: k خیلی کوچک به نویز حساس، k خیلی بزرگ مرزها را بیش‌ازحد صاف می‌کند.

۲.۲ کلاس‌های ما در فضای ویژگی

برای دیدنِ اینکه آیا کلاس‌ها اصلاً جداپذیرند، بردارهای ۹۰۰بُعدیِ HOG را با PCA به ۲ بُعد فشرده و رسم کردیم:



هر نقطه یک خودروی واقعی است؛ سه رنگ = سه مدل. کلاس‌ها زیاد هم‌پوشانی دارند

می‌بینید که کلاس‌ها به‌شدت درهم‌اند. این سه مدلِ خودرو واقعاً شبیهِ هم‌اند (هر سه سدانِ متوسطِ ایرانی)، پس این یک مسئلهٔ سخت است. این نمودار پیش‌بینی می‌کند که دقتِ بالا دشوار خواهد بود — و همین‌طور هم شد.

۳.۲ ویژگی‌های کلاسیکِ دیگر

HOG تنها ویژگیِ ممکن نیست. بسته به مسئله، ویژگی‌های دیگری هم داریم:

- **هیستوگرامِ رنگ:** توزیعِ رنگ‌ها را می‌شمارد. برای تفکیکِ اشیایی که رنگِ متفاوت دارند عالی است (مثلاً میوهٔ قرمز در برابرِ سبز)، اما به شکل بی‌اعتناست.
- **LBP (الگوی باینریِ محلی):** بافت را کد می‌کند؛ برای تشخیصِ بافتِ سطح (پارچه، دیوار، پوست) مفید است.
- **کیسهٔ کلماتِ بصری (Bag of Visual Words):** توصیف‌گرهای ORB/SIFT را (از هفتهٔ ۱۰) خوشه‌بندی می‌کند و هر تصویر را با هیستوگرامِ «کلمات بصری» توصیف می‌کند — پلی زیبا بینِ هفتهٔ ۱۰ و طبقه‌بندی.

نکتهٔ کلیدی

اغلب بهترین نتیجه از ترکیبِ چند ویژگی به دست می‌آید: مثلاً HOG (شکل) + هیستوگرامِ رنگ. این را «الحاقِ ویژگی» (feature concatenation) می‌گویند: دو بردار را به هم می‌چسبانیم و طبقه‌بند از هر دو استفاده می‌کند.

۳ تقسیمِ آموزش/آزمون: قاعدهٔ طلایی

مهم‌ترین اصلِ روش‌شناسیِ یادگیریِ ماشین: داده را به دو بخش تقسیم کن. روی بخشِ **آموزش** (train) طبقه‌بند را یاد بده، و روی بخشِ **آزمونِ** (test) دیده‌نشده ارزیابی کن.

هشدار / دام

هرگز دقت را روی داده‌ای که مدل دیده گزارش نکنید. مدل می‌تواند داده را حفظ کند و روی آن ۱۰۰٪ بگیرد، بی‌آنکه چیزی «یاد گرفته» باشد. آن عدد برای استفادهٔ واقعی بی‌معنی است.

ما ۷۰٪ آموزش و ۳۰٪ آزمون گرفتیم، به‌صورتِ **طبقه‌بندی‌شده** (stratified) — یعنی نسبتِ هر کلاس در هر دو بخش یکسان نگه داشته شد تا ارزیابی منصفانه باشد.

۴ نتایجِ واقعی و یک درسِ صادقانه

نتایجِ واقعیِ اجرا روی داده‌های شما:

طبقه‌بند	دقتِ آموزش	دقتِ آزمون
k-NN ($k=$)	۶۵٫۰	۴۳٫۰
SVM (کرنلِ RBF)	۱۰۰٫۰	۴۸٫۰

شانسِ تصادفی برای ۳ کلاس است، پس هر دو طبقه‌بند بهتر از شانس‌اند. اما به SVM دقت کنید: **۱۰۰٪ روی آموزش، ۴۸٪ روی آزمون**. این شکافِ بزرگ یک نام دارد: **بیش‌برازش** (overfitting).

۱.۴ بیش‌برازش چیست؟

بیش‌برازش یعنی مدل به‌جای یادگرفتنِ الگوی واقعی، ویژگی‌های تصادفی و نویزِ دادهٔ آموزش را حفظ کرده است.

- **نشانه:** دقتِ آموزش خیلی بیشتر از دقتِ آزمون (اینجا ۱۰۰٫۰ در برابرِ ۴۸٫۰).
- **درمان‌ها:** دادهٔ بیشتر، مدلِ ساده‌تر، منظم‌سازی (regularization)، یا ویژگی‌های بهتر و ناورداتر.

ریاضیات

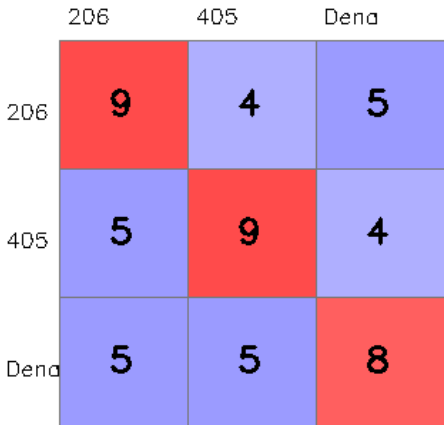
چرا SVM ما این‌قدر بد بیش‌برازش کرد؟ فقط ۱۲۶ بردارِ آموزش داشتیم، هر کدام ۹۰۰بُعدی — یعنی ابعادِ ویژگی بسیار بیشتر از تعدادِ نمونه‌هاست. یک RBF SVM با C بزرگ می‌تواند مرز را آن‌قدر خم کند که هر نقطهٔ آموزش را دقیق در بر بگیرد. نتیجه: ۱۰۰٫۱ روی آموزش، اما مرزی که به نویز کوک شده و روی آزمون فرو می‌ریزد. «ابعادِ بیشتر از نمونه‌ها» یک پرچمِ قرمز است.

نکتهٔ کلیدی

درمان‌های عملی برای همین مورد: کاهشِ C (منظم‌سازیِ بیشتر)، کاهشِ ابعاد با PCA پیش از طبقه‌بندی، یا جمع‌آوریِ دادهٔ آموزشِ بیشتر. مدلی که روی آموزش کامل و روی آزمون متوسط است، چیزِ اشتباهی یاد گرفته.

۵ ماتریسِ درهم‌ریختگی

یک عددِ دقت کافی نیست؛ باید بدانیم کدام کلاس‌ها اشتباه می‌شوند. **ماتریسِ درهم‌ریختگی** (confusion matrix) این را نشان می‌دهد:



سطر = کلاسِ واقعی، ستون = پیش‌بینی؛ قطر = درست، خارجِ قطر = خطاها

خانه‌های قطرِ اصلی نمونه‌های درست‌اند و خانه‌های خارجِ قطر خطاها. این ماتریس می‌گوید مثلاً چند تا ۲۰۶ اشتباهاً ۴۰۵ پیش‌بینی شده‌اند — اطلاعاتی بسیار مفیدتر از یک عددِ دقتِ کلی.

۱.۵ سنجه‌های فراتر از دقت

دقت (accuracy) می‌تواند گمراه‌کننده باشد، به‌خصوص وقتی کلاس‌ها نامتوازن‌اند.

ریاضیات

فرض کنید ۹۶٪ داده «سالم» و ۴٪ «معیوب» است. یک مدل که همیشه «سالم» بگوید ۹۶٪ دقت دارد اما هیچ عیبی را نمی‌گیرد — بی‌فایده! سنجه‌های بهتر:

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad \text{F}1 = \frac{2\,PR}{P+R}$$

صحت (precision): از مواردِ پیش‌بینی‌شده به‌عنوانِ مثبت، چندتا درست بود؟ **فراخوانی** (recall): از مواردِ واقعاً مثبت، چندتا را گرفتیم؟ **F۱** میانگینِ هارمونیکِ این دو است.

یادداشت

همیشه این سنجه‌ها را به‌ازای هر کلاس گزارش کنید، نه یک عددِ کلی. در سامانهٔ کنتورِ شما، «نگرفتنِ یک رقمِ اشتباه» (recall پایین) ممکن است بسیار پرهزینه‌تر از یک هشدارِ اضافی باشد.

۶ مصالحهٔ اریبی–واریانس

بیش‌برازش بخشی از یک تصویرِ بزرگ‌تر است به نامِ **مصالحهٔ اریبی–واریانس** (bias–variance tradeoff):

- **اریبیِ بالا (underfitting):** مدل بیش‌ازحد ساده است و حتی الگوی آموزش را هم نمی‌گیرد. نشانه: هم دقتِ آموزش و هم آزمون پایین.
- **واریانسِ بالا (overfitting):** مدل بیش‌ازحد پیچیده است و نویز را حفظ می‌کند. نشانه: دقتِ آموزش بالا، آزمون پایین (دقیقاً موردِ SVM ما).
- **نقطهٔ مطلوب:** پیچیدگیِ متعادل که هم الگو را می‌گیرد و هم تعمیم می‌یابد.

ریاضیات

با افزایشِ پیچیدگیِ مدل، خطای آموزش پیوسته کاهش می‌یابد، اما خطای آزمون ابتدا کم و سپس دوباره زیاد می‌شود (منحنیِ U). هدفِ ما پیداکردنِ کفِ آن U است — نه ساده‌ترین و نه پیچیده‌ترین مدل. ابزارهایی مثلِ اعتبارسنجیِ متقابل به یافتنِ این نقطه کمک می‌کنند.

یادداشت

برای مسئلهٔ ما: SVM با $C =$ در سمتِ واریانسِ بالا بود. کاهشِ C یا کاهشِ ابعادِ ویژگی، مدل را به سمتِ کفِ U می‌بَرَد. این‌ها آزمایش‌هایی هستند که در پروژهٔ این هفته انجام می‌دهید.

۷ اعتبارسنجیِ متقابل: خودت را گول نزن

یک تقسیمِ واحدِ آموزش/آزمون نویزی است؛ ممکن است تصادفاً خوش‌شانس یا بدشانس باشید. **اعتبارسنجیِ متقابلِ تایی–k** (k-fold cross-validation) داده را به k بخش تقسیم می‌کند، k بار آموزش/آزمون می‌گیرد (هر بار یک بخشِ متفاوت به‌عنوانِ آزمون) و میانگین می‌گیرد — برآوردی پایدارتر.

هشدار / دام

پارامترها را روی مجموعهٔ **اعتبارسنجی** (validation) یا با CV تنظیم کنید، هرگز روی مجموعهٔ آزمون. مجموعهٔ آزمون فقط یک بار، در انتهای کار، استفاده می‌شود. اگر بارها به آن نگاه کنید و پارامتر تنظیم کنید، عملاً آن را هم به بخشی از آموزش تبدیل کرده‌اید و دقتِ گزارش‌شده متورم می‌شود.

۸ نرمال‌سازیِ ویژگی: گامی که فراموش می‌شود

یک نکتهٔ عملیِ مهم: بسیاری از طبقه‌بندها (به‌ویژه k-NN و SVM) به مقیاسِ ویژگی‌ها حساس‌اند. اگر یک ویژگی در بازهٔ ۰ تا ۱۰۰۰ و دیگری در بازهٔ ۰ تا ۱ باشد، فاصلهٔ اقلیدسی عملاً فقط ویژگیِ اول را می‌بیند.

ریاضیات

راه‌حل، **استانداردسازی** است: هر ویژگی را طوری تبدیل کنیم که میانگینِ صفر و انحرافِ‌معیارِ یک داشته باشد:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

که μ_j و σ_j میانگین و انحرافِ‌معیارِ ویژگیِ j روی دادهٔ آموزش‌اند. نکتهٔ حیاتی: این μ و σ را فقط از آموزش حساب کنید و همان‌ها را روی آزمون اعمال کنید — وگرنه اطلاعاتِ آزمون به آموزش نشت می‌کند.

هشدار / دام

یک اشتباهِ بسیار رایج: محاسبهٔ میانگین/انحرافِ‌معیار روی کلِ داده پیش از تقسیم. این «نشتِ داده» (data leakage) است و دقتِ گزارش‌شده را به‌طورِ کاذب بالا می‌برد. همیشه اول تقسیم کنید، بعد نرمال‌سازی را روی آموزش برازش دهید.

خبرِ خوب برای مورد ما: HOG به‌طورِ درونی بلوک‌هایش را نرمال می‌کند، پس مقیاسِ ویژگی‌ها از قبل نسبتاً یکنواخت است. اما برای ویژگی‌های ترکیبی (HOG + رنگ) استانداردسازی ضروری می‌شود.

۹ ژرف‌نگری: مهندسیِ ویژگی در برابرِ یادگیریِ ویژگی

امروز ما ویژگی‌ها را مهندسی کردیم (HOG) و فقط طبقه‌بند را یاد دادیم. سقفِ ۴۸٪ روی خودروهای شبیه‌به‌هم، محدودیتِ ویژگیِ دست‌طراحی را نشان می‌دهد: HOG شکلِ کلی را می‌گیرد اما تفاوت‌های ظریفِ بینِ ۲۰۶ و ۴۰۵ را خوب کد نمی‌کند.

نکتهٔ کلیدی

یادگیریِ عمیق (هفته‌های ۱۳ و ۱۴) این را تغییر می‌دهد: خودِ ویژگی‌ها را مستقیماً از پیکسل‌ها یاد می‌گیرد. به‌جای اینکه ما تصمیم بگیریم «جهتِ گرادیان مهم است»، شبکه خودش کشف می‌کند کدام الگوها برای تفکیکِ کلاس‌ها مفیدند. همین، جهشِ دقت در مسائلِ سخت را ممکن می‌کند. پس این جلسه هم یک ابزارِ عملی به شما داد و هم انگیزهٔ هفته‌های بعد را روشن کرد.

۱.۹ چطور ۴۸٪ را بهبود دهیم؟ (یک نقشهٔ آزمایش)

سقفِ ۴۸٪ پایانِ راه نیست؛ چند آزمایشِ مشخص که در پروژه امتحان می‌کنید:

- **دادهٔ بیشتر:** از ۶۰ به ۳۰۰ تصویر در هر کلاس. معمولاً بیشترین تأثیر را دارد و شکافِ بیش‌برازش را کم می‌کند.
- **کاهشِ C:** از ۱۰ به ۱ یا ۰٫۱. مرزِ SVM را ساده‌تر می‌کند؛ دقتِ آموزش پایین می‌آید اما آزمون معمولاً بالا می‌رود.
- **PCA پیش از طبقه‌بندی:** ۹۰۰ بُعد را به مثلاً ۵۰ بُعد کاهش دهید؛ نویز کم و بیش‌برازش مهار می‌شود.
- **ویژگیِ ترکیبی:** HOG + هیستوگرامِ رنگ. اگر مدل‌ها رنگِ رایجِ متفاوت داشته باشند، کمک می‌کند.
- **اعتبارسنجیِ متقابل:** به‌جای یک تقسیم، ۵-تایی؛ برآوردِ دقت پایدارتر و قابلِ اعتمادتر می‌شود.

نکتهٔ کلیدی

روشِ مهندسی: هر بار یک چیز را تغییر دهید و اثرش را روی دقتِ آزمون بسنجید. تغییرِ هم‌زمانِ چند چیز، فهمِ اینکه چه چیزی کمک کرد را غیرممکن می‌کند. این «آزمایشِ کنترل‌شده» هستهٔ کارِ تجربی در یادگیریِ ماشین است.

۲.۹ پیوند با DocScan

DocScan باید ارقام را بخواند. یک رقم یک تصویرِ کوچک است و یک طبقه‌بند آن را به یکی از ۰ تا ۹ می‌نگارد. دقیقاً همین خط‌لولهٔ «ویژگی → طبقه‌بند» بود که OCR کلاسیک با آن کار می‌کرد. هفتهٔ آینده ویژگی‌های دستی را با یک **CNN** که خودش ویژگی‌ها را یاد می‌گیرد جایگزین می‌کنیم و خواهیم دید دقت چطور جهش می‌کند.

۱۰ کدِ کاملِ خط‌لوله

```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
X = [hog(gray(img)) for img in images]      # 1) features
Xtr, Xte, ytr, yte = train_test_split(      # 2) split
        X, y, test_size3.0=, stratify=y, random_state=0)
clf = SVC(kernel='rbf', C=10).fit(Xtr, ytr)  # 3) learn
pred = clf.predict(Xte)                       # 4) evaluate
print(accuracy_score(yte, pred))              # HONEST score
print(confusion_matrix(yte, pred))
```

ترتیب همیشه ثابت است: استخراجِ ویژگی → تقسیم → برازش → ارزیابی روی دادهٔ دیده‌نشده.

۱۱ تمرینِ کلاسی و پاسخ

تمرین

۱) SVM ما آموزش ۱٫۰۰ و آزمون ۰٫۴۸ گرفت. نامِ مشکل و دو راهِ کاهشش چیست؟

۲) یک آشکارسازِ عیب ۹۶٪ دقت گزارش می‌کند اما عیب‌ها ۴٪ داده‌اند. چرا ممکن است بی‌فایده باشد؟

۳) می‌خواهید ۲۰۶ را از ۴۰۵ تفکیک کنید. چه ویژگی‌ای غیر از HOG می‌تواند کمک کند؟

پاسخ‌ها: (۱) **بیش‌برازش**؛ دادهٔ بیشتر جمع کنید، یا SVM را ساده‌تر/منظم‌تر کنید (کاهشِ C)، یا ابعاد را با PCA کم کنید. (۲) پیش‌بینیِ همیشه«سالم» ۹۶٪ می‌گیرد اما **فراخوانیِ صفر** روی عیب‌ها دارد — دقت سنجهٔ اشتباهی است؛ باید precision/recall گزارش شود. (۳) **هیستوگرامِ رنگ** (مدل‌ها رنگِ رایجِ متفاوت دارند)، اندازه/نسبتِ ابعاد، یا ویژگی‌های ناحیهٔ آرم.

۱۲ زمینهٔ تاریخی

یادگیریِ ماشین در بینایی یک‌شبه نیامد. در دهه‌های ۱۹۹۰ و ۲۰۰۰، ترکیب‌هایی مثلِ HOG+SVM یا Haar+AdaBoost (آشکارسازِ چهرهٔ Viola–Jones در ۲۰۰۱) استانداردِ صنعت بودند و در دوربین‌ها و خودروها به کار می‌رفتند. نقطهٔ عطف در سالِ ۲۰۱۲ رخ داد: شبکهٔ AlexNet در مسابقهٔ ImageNet با اختلافِ زیادی همهٔ روش‌های مبتنی‌برویژگیِ دستی را شکست داد و نشان داد که یادگرفتنِ ویژگی‌ها از پیکسل‌ها بسیار قدرتمندتر است. از آن پس، تقریباً همهٔ سامانه‌های شناساییِ پیشرفته به یادگیریِ عمیق کوچ کردند.

یادداشت

اما — و این مهم است — روش‌های کلاسیکِ این جلسه هنوز جای خود را دارند: وقتی داده کم است، وقتی به سرعتِ بالا و منابعِ محدود نیاز دارید، یا وقتی قابلیتِ تفسیر مهم است. یک مهندسِ خوب هر دو ابزار را می‌شناسد و ابزارِ مناسبِ مسئله را انتخاب می‌کند، نه صرفاً جدیدترین را.

۱۳ جمع‌بندی

در این جلسه یاد گرفتیم که خط‌لولهٔ کلاسیکِ شناسایی «ویژگی → طبقه‌بند» است؛ HOG شکل را به بردار تبدیل می‌کند؛ k-NN و SVM مرزِ تصمیم یاد می‌گیرند؛ تقسیمِ آموزش/آزمونِ طبقه‌بندی‌شده ما را صادق نگه می‌دارد؛ بیش‌برازش (آموزش ≫ آزمون) یک خطرِ واقعی است که روی داده‌های خودمان دیدیم؛ و ماتریسِ درهم‌ریختگی و سنجه‌های precision/recall بسیار آموزنده‌تر از یک عددِ دقت‌اند. مهم‌تر از همه، سقفِ ۴۸٪ روی خودروهای شبیه‌به‌هم به ما نشان داد که ویژگیِ دست‌طراحی حدی دارد — و همین، انگیزهٔ رفتن به سراغِ یادگیریِ عمیق در هفته‌های ۱۳ و ۱۴ است.